

Práctica 3 — Parte 2: Análisis de las gramáticas de números romanos

Pareja 6: Juan Herrero y Santiago Fernández

10 de diciembre de 2025

Gramática (extracto)

Fichero: `src/parser.out` (reglas numeradas):

```
0 S' -> romanNumber
1 romanNumber -> thousand hundred ten digit
2 thousand -> M M M
3 thousand -> M M
4 thousand -> M
5 thousand -> lambda
6 small_hundred -> C C C
7 small_hundred -> C C
8 small_hundred -> C
9 hundred -> C M
10 hundred -> C D
11 hundred -> D small_hundred
12 hundred -> small_hundred
13 hundred -> lambda
14 small_ten -> X X X
15 small_ten -> X X
16 small_ten -> X
17 ten -> X C
18 ten -> X L
19 ten -> L small_ten
20 ten -> small_ten
21 ten -> lambda
22 small_digit -> I I I
23 small_digit -> I I
24 small_digit -> I
25 digit -> I X
26 digit -> I V
27 digit -> V small_digit
28 digit -> small_digit
29 digit -> lambda
30 lambda -> <empty>
31 roman -> romanNumber
```

1. Conjuntos PRIMERO y SIGUIENTE

Se muestran los conjuntos calculados a partir de las producciones anteriores.

PRIMERO (FIRST)

- $FIRST(\text{thousand}) = \{M, \epsilon\}$
- $FIRST(\text{small_hundred}) = \{C\}$
- $FIRST(\text{hundred}) = \{C, D, \epsilon\}$
- $FIRST(\text{small_ten}) = \{X\}$
- $FIRST(\text{ten}) = \{X, L, \epsilon\}$
- $FIRST(\text{small_digit}) = \{I\}$
- $FIRST(\text{digit}) = \{I, V, \epsilon\}$
- $FIRST(\text{lambda}) = \{\epsilon\}$
- $FIRST(\text{romanNumber}) = \{M, C, D, X, L, I, V, \epsilon\}$
- $FIRST(\text{roman}) = FIRST(\text{romanNumber})$

SIGUIENTE (FOLLOW)

Tomando como símbolo inicial **romanNumber** (la tabla generada por PLY usa **S' -> romanNumber**) y aplicando las reglas de cálculo iterativo:

- $FOLLOW(\text{romanNumber}) = \{\$ \}$
- $FOLLOW(\text{thousand}) = \{C, D, X, L, I, V, \$ \}$
- $FOLLOW(\text{hundred}) = \{X, L, I, V, \$ \}$
- $FOLLOW(\text{ten}) = \{I, V, \$ \}$
- $FOLLOW(\text{digit}) = \{\$ \}$
- $FOLLOW(\text{small_hundred}) = FOLLOW(\text{hundred}) = \{X, L, I, V, \$ \}$
- $FOLLOW(\text{small_ten}) = FOLLOW(\text{ten}) = \{I, V, \$ \}$
- $FOLLOW(\text{small_digit}) = FOLLOW(\text{digit}) = \{\$ \}$
- $FOLLOW(\text{lambda})$ (símbolo de producción vacía): se usa cuando aparece en posiciones donde se acepta el siguiente terminal; su FOLLOW será el conjunto correspondiente de la posición en la que aparece (por ejemplo, $FOLLOW(\text{lambda})$ contiene $\{C, D, X, L, I, V, \$ \}$ cuando aparece en la posición de **thousand**).

Comentarios: los conjuntos han sido deducidos de las producciones y de la observación de que muchas componentes admiten ϵ (por ejemplo **thousand**, **hundred**, **ten**, **digit**), con lo que los conjuntos FIRST se propagan a la posición siguiente y los conjuntos FOLLOW incluyen el operador fin de cadena \$.

2. Estado inicial S0 del autómata LR(1) y sus transiciones

A partir de **src/parser.out** se extrae el estado 0 (S0) tal cual lo genera PLY. Se incluye aquí el listado de items y las transiciones salientes:

Estado S0 (items)

```
(0) S' -> . romanNumber
(1) romanNumber -> . thousand hundred ten digit
(2) thousand -> . M M M
(3) thousand -> . M M
(4) thousand -> . M
(5) thousand -> . lambda
(30) lambda -> .
```

Transiciones desde S0

■ Acciones (terminales):

```
M      : shift and go to state 3
C, D, X, L, I, V, $end : reduce using rule 30 (lambda -> .)
```

■ Gotos (no terminales):

```
romanNumber : shift and go to state 1
thousand    : shift and go to state 2
lambda      : shift and go to state 4
```

Observación: en S0, para muchas terminales distintas de 'M' la acción es reducir por la producción de la no terminal 'lambda' (regla 30), porque desde el punto de vista del parser la producción 'thousand -> lambda' está disponible y el lookahead determina reducción.

3. Tabla LR(1) para S0 (filtrada) y detección de conflictos

Mostramos la fila correspondiente al estado S0 para las columnas relevantes (terminales y gotos):

Entrada	Acción en S0
M	shift, goto estado 3
C	reduce por regla 30 (lambda ->)
D	reduce por regla 30 (lambda ->)
X	reduce por regla 30 (lambda ->)
L	reduce por regla 30 (lambda ->)
I	reduce por regla 30 (lambda ->)
V	reduce por regla 30 (lambda ->)
\$	reduce por regla 30 (lambda ->)
Goto(thousand)	estado 2
Goto(romanNumber)	estado 1
Goto(lambda)	estado 4

Dado que en S0 no existe ninguna celda que contenga simultáneamente una acción de shift y otra de reduce para la misma terminal (por ejemplo, para M hay shift; para C hay reduce y no shift), en S0 no se detectan conflictos shift/reduce ni reduce/reduce. Por tanto, considerando únicamente S0, este estado es compatible con un autómata LR(1) sin conflictos.

4. Efecto en el proceso de análisis y ejemplos

Efecto general

La presencia de producciones que admiten ϵ provoca que, en los estados iniciales, el parser tenga reducciones por ϵ ante múltiples símbolos de lookahead; esto es normal y, si las reducciones

no confligen con shift (como en S0), no aparece conflicto. En lenguajes con múltiples alternativas que comienzan por el mismo prefijo, el lookahead evita ambigüedades en LR(1).

Ejemplos

1. Cadena vacía (EOF inmediatamente): - En S0 el lookahead \$ provoca reducción por `lambda` en la posición `thousand` $\rightarrow$ `lambda`, y finalmente la derivación vacía se considera válida (el parser acepta la cadena vacía). Este comportamiento viene de permitir que todas las componentes (`thousand,hundred,ten,digit`) deriven ϵ .
2. Entrada que comienza por M: `M$` - En S0 con M el parser hace shift y entra en la construcción de `thousand` (estado 3), lo que evita reducir por `lambda` en esa posición; esto es correcto puesto que M indica la presencia de miles.
3. Entrada que comienza por C (por ejemplo `C$`): - En S0 con C se aplica la reducción por `lambda` en la producción `thousand` $\rightarrow$ `lambda` (porque C no es M), y el parser seguirá analizando `hundred` a partir del siguiente estado apropiado. Esto muestra cómo el lookahead C permite distinguir entre tener `thousand` vacío o tener `thousand` que comience por M.